

Persistencia de la Quantum Wallet con SQLite

Del modelo de clases UML al esquema relacional de Quantum Core

Diseño e implementación de una base de datos relacional con llaves primarias y foráneas, tipos de datos coherentes y restricciones de integridad, generada de forma automatizada desde Python

ESTUDIANTE	Deibis Zuluaga Baena
NÚCLEO	Fundamentos de Software · FUNSO_2_N_H
ACTIVIDAD	Introducción a las bases de datos
HERRAMIENTAS	SQLite 3 · Python 3.14 (módulo sqlite3) · DB Browser for SQLite
ENTREGABLES	configurar_db.py, quantum_wallet.db, modelo entidad-relación (PNG, SVG y .drawio) e informe
DOCENTE	Simón Peláez Loaiza
INSTITUCIÓN	CEIPA Business School
FECHA DE ENTREGA	4 de octubre de 2026

Objetivo del trabajo

Objetivo: transformar el modelo de clases de la Quantum Wallet (`Usuario` , `UsuarioEmpresa` y `wallet`) en un esquema de base de datos relacional en SQLite que garantice la persistencia de la información de Quantum Core, mediante tablas con llaves primarias y foráneas, tipos de datos coherentes y restricciones de integridad, y automatizar su creación con el script `configurar_db.py` .

CÓDIGO ORIGINAL DE LA ACTIVIDAD

El enunciado de la actividad y el repositorio del curso definen dos tablas: `usuarios` , con nombre, correo electrónico y NIT, y `wallets` , con el saldo y la llave foránea `id_propietario` , junto con una inserción de prueba. Ese diseño se conserva completo. Para enriquecer el ejercicio, el esquema se amplió hasta cubrir todas las clases del modelo UML de la Quantum Wallet, incluidas `Transaccion` y `Empleado` , la agregación entre empresas y empleados, y la configuración global del Banco Central. Los elementos adicionales se identifican en el código con el comentario `[AGREGADO]` y en el modelo entidad-relación con fondo amarillo.

ALCANCE DEL ESQUEMA

La base de datos resultante está compuesta por seis tablas, cuatro triggers, tres vistas y dos índices. Además de almacenar la información, el esquema traslada a la base de datos las reglas del modelo de clases: la composición entre el usuario y su billetera, la herencia de los empleados, la agregación entre empresas y empleados, el límite por transacción y la instancia única de la configuración global.

CRITERIO PARA LOS TIPOS DE DATOS

SQLite almacena los valores en clases de almacenamiento. Se asignó `INTEGER` a los identificadores; `REAL` a los valores monetarios (saldo, monto, salario y límite), con el fin de conservar los centavos; y `TEXT` a nombres, documentos, códigos y fechas. La cédula y el NIT se definieron como texto porque son identificadores y no cantidades: pueden contener guiones o ceros a la izquierda. Las fechas se registran en formato ISO 8601 (`AAAA-MM-DD HH:MM:SS`), que SQLite ordena y compara de forma correcta.

Instalación y verificación de SQLite

La persistencia se implementó con SQLite, un motor de base de datos relacional que almacena la base completa en un único archivo y no requiere un servidor. El entorno se preparó en macOS con los componentes que se describen a continuación.

COMPONENTE	VERSIÓN	FUNCIÓN EN EL PROYECTO
DB Browser for SQLite	Aplicación de escritorio	Herramienta gráfica de código abierto instalada para administrar SQLite. Se utilizó para abrir <code>quantum_wallet.db</code> e inspeccionar su estructura y sus datos.
Consola <code>sqlite3</code>	SQLite 3.43.2	Intérprete de línea de comandos incluido en macOS, ubicado en <code>/usr/bin/sqlite3</code> .
Python y módulo <code>sqlite3</code>	Python 3.14.7 · SQLite 3.50.4	Biblioteca estándar de Python con la que <code>configurar_db.py</code> crea la base, inserta los datos y ejecuta las verificaciones.
Visual Studio Code	—	Editor del código fuente y del modelo entidad-relación, este último con la extensión Draw.io Integration.

Verificación de la instalación

La instalación se comprobó desde la Terminal con la ubicación y la versión de la consola `sqlite3`, la versión de Python y la importación del módulo `sqlite3`, que imprime la versión de la biblioteca con la que trabaja el script.

```

/usr/bin/sqlite3
3.43.2 2023-10-10 13:08:14 1b37c146ee9ebb7acd0160c0ab1fd11017a419fa8a3187386ed8cb32b709aapl (64-bit)
Python 3.14.7
SQLite listo para Python - 3.50.4
zulu@MacBook-Air-de-Deibis ~ %

```

Figura 1. Verificación de SQLite en la Terminal de macOS: consola `sqlite3` 3.43.2, Python 3.14.7 y módulo `sqlite3` con SQLite 3.50.4.

VERSIONES DEL MOTOR

La consola y Python reportan versiones distintas porque cada uno incorpora su propia copia de la biblioteca SQLite. Ambas comparten el mismo formato de archivo, por lo que la base creada desde Python se abre sin cambios desde la consola y desde DB Browser for SQLite. Python no requiere instalaciones adicionales: el módulo `sqlite3` forma parte de su biblioteca estándar.

Identificación de las entidades

Cada clase del modelo UML con identidad y datos propios se convirtió en una tabla. Las relaciones de herencia, composición y agregación se resolvieron mediante llaves foráneas y restricciones.

CLASE	TABLA	ORIGEN	CRITERIO DE DISEÑO
Usuario UsuarioEmpresa	usuarios	Original	Ambas clases comparten identidad y atributos; la subclase solo agrega el NIT. Se almacenan en una tabla con la columna tipo_usuario , lo que evita duplicar datos y uniones innecesarias.
Wallet	wallets	Original	Concentra el dinero del usuario y se vincula a su propietario mediante id_propietario .
Transaccion	transacciones	Adicionada	El historial es una lista dentro de wallet . En el modelo relacional, una lista se representa como una tabla con una fila por movimiento.
Empleado	empleados	Adicionada	Agrega cargo y salario. Se almacena en una tabla propia cuya llave primaria es, a la vez, llave foránea a usuarios , para no dejar columnas vacías en los demás tipos de usuario.
UsuarioEmpresa ◇ Empleado	vinculaciones	Adicionada	La agregación es de muchos a muchos: una empresa agrupa varios empleados y un empleado puede vincularse a varias empresas. Esa relación requiere una tabla intermedia.
BancoCentral	configuracion_sistema	Adicionada	Los parámetros globales (moneda y límite por transacción) se leen desde la base. La tabla admite una sola fila, del mismo modo que el Singleton admite una sola instancia.

Decisiones de diseño

DECISIÓN	JUSTIFICACIÓN
Herencia resuelta en dos formas	Usuario y UsuarioEmpresa comparten la tabla usuarios porque la diferencia es una sola columna. Empleado se separa porque agrega dos atributos obligatorios que no aplican a los demás usuarios. En ambos casos se evitan columnas vacías y datos repetidos.
Identificador de la billetera	En la clase, id_billetera se genera con id(self) , una dirección de memoria que cambia en cada ejecución y no sirve como dato persistente. Se reemplazó por un código estable (W-00001) derivado de la llave primaria.
Contador de transacciones	El atributo estático _contador de Transaccion se reemplazó por AUTOINCREMENT : la base asigna el número y lo conserva entre ejecuciones.
Listas de objetos	Las listas historial y empleados se reemplazaron por tablas relacionadas mediante llaves foráneas.
Métodos de las clases	Los métodos no se almacenan. Los que modifican datos (recargar() , realizar_pago() , transferir() y pagar_nomina()) se implementaron como funciones de configurar_db.py que ejecutan las sentencias SQL dentro de una transacción.

Esquema relacional de quantum_wallet.db

Quantum Wallet · Modelo entidad-relación

Esquema relacional de quantum_wallet.db (SQLite) derivado del diagrama de clases UML de usuarios.py

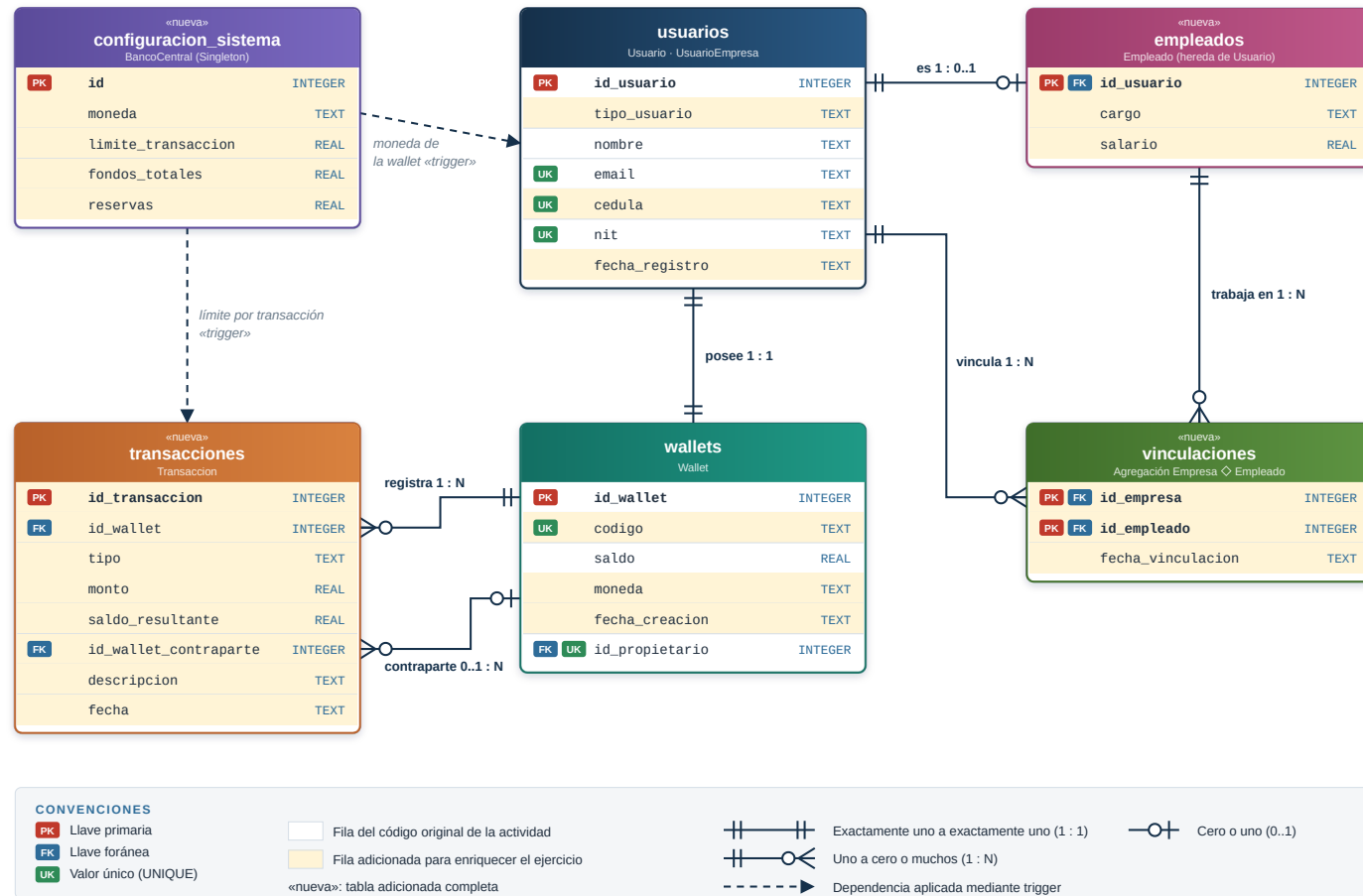


Figura 2. Modelo entidad-relación de la Quantum Wallet en notación de pata de gallo. Fondo blanco: código original de la actividad; fondo amarillo: elementos adicionados para enriquecer el ejercicio. Elaborado en formato .drawio con Draw.io Integration para Visual Studio Code.

Estructura de cada tabla

Las tablas siguientes detallan los campos, su tipo y sus restricciones. Las filas con fondo amarillo corresponden a los campos adicionados para enriquecer el ejercicio.

usuarios

CAMPO	TIPO	RESTRICCIONES	PROPÓSITO
id_usuario	INTEGER	PRIMARY KEY AUTOINCREMENT	Identificador único; no se reutiliza al eliminar un registro.
tipo_usuario	TEXT	NOT NULL; CHECK: PERSONA, EMPRESA o EMPLEADO	Indica la clase de origen del registro.
nombre	TEXT	NOT NULL; CHECK: no vacío	Nombre de la persona o razón social de la empresa.
email	TEXT	UNIQUE; CHECK: formato de correo	Correo de contacto, opcional como en el constructor de la clase.
cedula	TEXT	UNIQUE; obligatoria para empleados	Documento de identidad de personas y empleados.
nit	TEXT	UNIQUE; obligatorio solo para empresas	Identificación tributaria de las empresas.
fecha_registro	TEXT	NOT NULL; DEFAULT fecha actual	Trazabilidad del alta del usuario.

wallets

CAMPO	TIPO	RESTRICCIONES	PROPÓSITO
id_wallet	INTEGER	PRIMARY KEY AUTOINCREMENT	Identificador único de la billetera.
codigo	TEXT	NOT NULL; UNIQUE	Código visible de la billetera (W-00001).
saldo	REAL	NOT NULL; DEFAULT 0; CHECK saldo >= 0	Dinero disponible, con centavos. La restricción impide saldos negativos.
moneda	TEXT	NOT NULL; DEFAULT 'COP'; CHECK: 3 caracteres	Código de moneda según la norma ISO 4217.
fecha_creacion	TEXT	NOT NULL; DEFAULT fecha actual	Fecha de apertura de la billetera.
id_propietario	INTEGER	NOT NULL; UNIQUE; FK → usuarios ON DELETE CASCADE	Dueño de la billetera. UNIQUE garantiza la relación uno a uno.

transacciones

CAMPO	TIPO	RESTRICCIONES	PROPÓSITO
id_transaccion	INTEGER	PRIMARY KEY AUTOINCREMENT	Número consecutivo del movimiento.
id_wallet	INTEGER	NOT NULL; FK → wallets ON DELETE CASCADE	Billetera a la que pertenece el movimiento.

tipo	TEXT	CHECK: RECARGA, PAGO, TRANSFERENCIA_ENVIADA o TRANSFERENCIA_RECIBIDA	Naturaleza del movimiento; determina si suma o resta.
monto	REAL	NOT NULL; CHECK monto > 0	Valor del movimiento, siempre positivo.
saldo_resultante	REAL	NOT NULL; CHECK >= 0	Saldo de la billetera después del movimiento, como en la clase <code>Transaccion</code> .
id_wallet_contraparte	INTEGER	FK → wallets ON DELETE SET NULL; distinta de id_wallet	Billetera de origen o de destino en las transferencias.
descripcion	TEXT	—	Concepto del movimiento.
fecha	TEXT	NOT NULL; DEFAULT fecha actual	Fecha y hora del movimiento.

empleados

CAMPO	TIPO	RESTRICCIONES	PROPÓSITO
id_usuario	INTEGER	PRIMARY KEY; FK → usuarios ON DELETE CASCADE	El empleado es un usuario: comparte su identificador.
cargo	TEXT	NOT NULL	Cargo dentro de la empresa.
salario	REAL	NOT NULL; CHECK salario > 0	Valor que se transfiere en el pago de la nómina.

vinculaciones

CAMPO	TIPO	RESTRICCIONES	PROPÓSITO
id_empresa	INTEGER	PRIMARY KEY compuesta; FK → usuarios ON DELETE CASCADE	Empresa que vincula al empleado.
id_empleado	INTEGER	PRIMARY KEY compuesta; FK → empleados ON DELETE CASCADE	Empleado vinculado. La llave compuesta impide registrar dos veces el mismo vínculo.
fecha_vinculacion	TEXT	NOT NULL; DEFAULT fecha actual	Fecha de inicio del vínculo.

configuracion_sistema

CAMPO	TIPO	RESTRICCIONES	PROPÓSITO
id	INTEGER	PRIMARY KEY; CHECK id = 1	Solo admite una fila: la configuración es única.
moneda	TEXT	NOT NULL; DEFAULT 'COP'	Moneda oficial que reciben las nuevas billeteras.
limite_transaccion	REAL	NOT NULL; CHECK > 0	Monto máximo permitido por movimiento (50.000.000).
fondos_totales	REAL	NOT NULL; DEFAULT 0	Fondos del Banco Central.
reservas	REAL	NOT NULL; DEFAULT 0	Reservas emitidas por el Banco Central.

Relaciones, integridad y normalización

Las relaciones entre tablas se implementaron con llaves foráneas. El comportamiento al eliminar un registro principal reproduce el tipo de relación definido en el modelo de clases.

RELACIÓN	CARDINALIDAD	IMPLEMENTACIÓN	AL ELIMINAR EL REGISTRO PRINCIPAL
usuarios – wallets	1 : 1	FK <code>id_propietario</code> con NOT NULL y UNIQUE	Se elimina la billetera (CASCADE). Corresponde a la composición: la billetera no existe sin su dueño.
wallets – transacciones	1 : N	FK <code>id_wallet</code>	Se elimina el historial (CASCADE), también por composición.
wallets – transacciones (contraparte)	0..1 : N	FK <code>id_wallet_contraparte</code>	El movimiento se conserva y la referencia queda nula (SET NULL), para no alterar el historial de la otra billetera.
usuarios – empleados	1 : 0..1	Llave primaria compartida	Se elimina la ficha del empleado (CASCADE).
empresas – empleados	N : M	Tabla <code>vinculaciones</code> con llave compuesta	Se elimina el vínculo, pero el empleado se conserva. Corresponde a la agregación: la parte existe por fuera del todo.

ACTIVACIÓN DE LAS LLAVES FORÁNEAS

SQLite no verifica las llaves foráneas por defecto. Por esa razón, `configurar_db.py` ejecuta `PRAGMA foreign_keys = ON;` en cada conexión; sin esa instrucción, las restricciones de la tabla serían solo declarativas.

Reglas aplicadas mediante triggers

Algunas reglas del modelo de clases involucran más de una tabla y no pueden expresarse con una restricción de columna. Para ellas se definieron triggers, que la base de datos ejecuta automáticamente antes o después de una inserción.

TRIGGER	MOMENTO	REGLA
<code>trg_usuario_crea_wallet</code>	Después de insertar un usuario	Crea su billetera con el código y la moneda del sistema, del mismo modo que el constructor de <code>Usuario</code> crea su <code>Wallet</code> .
<code>trg_transaccion_valida_limite</code>	Antes de insertar una transacción	Rechaza montos superiores al límite configurado; equivale a <code>BancoCentral.validar_monto()</code> .
<code>trg_empleado_valida_tipo</code>	Antes de insertar un empleado	Exige que el usuario sea de tipo EMPLEADO.
<code>trg_vinculacion_valida_empresa</code>	Antes de insertar un vínculo	Exige que quien vincula sea una EMPRESA.

Vistas e índices

Se definieron tres vistas que almacenan consultas de uso frecuente: `vista_saldos` (usuario con su billetera y saldo), `vista_nomina` (empresas con sus empleados, cargos y salarios) y `vista_movimientos` (historial con el nombre del titular). Se crearon además dos índices: `idx_transacciones_wallet_fecha`, para consultar el historial de una billetera en orden cronológico, e `idx_vinculaciones_empleado`, para buscar las empresas de un empleado.

Normalización

Primera forma normal. Todos los campos contienen valores atómicos. Las listas `historial` y `empleados` se trasladaron a tablas propias en lugar de almacenarse en una sola columna.

Segunda forma normal. La única llave compuesta del esquema está en `vinculaciones`, y su único campo adicional, `fecha_vinculacion`, depende de la llave completa: describe el vínculo, no la empresa ni el empleado por separado.

Tercera forma normal. Los datos del propietario no se repiten en `wallets` ni en `transacciones`; se obtienen mediante `JOIN` a partir de las llaves foráneas.

DESNORMALIZACIÓN CONTROLADA DEL SALDO

El saldo podría calcularse a partir del historial de transacciones. Se mantiene como columna porque es el atributo principal de `wallet` y la consulta más frecuente del sistema. Su consistencia se garantiza porque cada movimiento actualiza el saldo y registra la transacción dentro de una misma transacción de base de datos; además, el script concilia ambos valores al finalizar.

El script configurar_db.py

El script crea la base de datos desde cero, inserta los datos de prueba y ejecuta las verificaciones. Su estructura se organiza en los bloques siguientes.

BLOQUE

FUNCIÓN

ESQUEMA_SQL	Sentencias SQL de creación de tablas, índices, triggers y vistas.
conectar() , crear_esquema()	Abren la conexión con las llaves foráneas activas y recrean el archivo en cada ejecución.
registrar_usuario() , registrar_empleado() , vincular_empleado()	Registran usuarios, empleados y vínculos con parámetros seguros (?).
recargar() , realizar_pago() , transferir() , pagar_nomina()	Equivalen a los métodos de las clases; cada una actualiza el saldo y registra el movimiento de forma atómica.
insertar_datos_prueba()	Escenarios del código original (Ana Torres, Bancolombia) y de la versión ampliada (Quantum SAS).
verificar() , probar_integridad() , probar_autoincrement_y_cascada()	Comprueban la estructura, la conciliación de saldos, las restricciones y AUTOINCREMENT .

Definición de la tabla wallets y del trigger de composición

```
CREATE TABLE wallets (
  id_wallet      INTEGER PRIMARY KEY AUTOINCREMENT,
  codigo         TEXT NOT NULL UNIQUE,           -- [AGREGADO] id_billetera
  saldo          REAL NOT NULL DEFAULT 0
  CONSTRAINT ck_wallet_saldo_no_negativo CHECK (saldo >= 0),
  moneda         TEXT NOT NULL DEFAULT 'COP'     -- [AGREGADO]
  CONSTRAINT ck_wallet_moneda CHECK (length(moneda) = 3),
  fecha_creacion TEXT NOT NULL DEFAULT (datetime('now', 'localtime')), -- [AGREGADO]
  id_propietario INTEGER NOT NULL UNIQUE,
  FOREIGN KEY (id_propietario) REFERENCES usuarios(id_usuario)
  ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TRIGGER trg_usuario_crea_wallet
AFTER INSERT ON usuarios
BEGIN
  INSERT INTO wallets (codigo, moneda, id_propietario)
  VALUES ('W-' || printf('%05d', NEW.id_usuario),
    (SELECT moneda FROM configuracion_sistema WHERE id = 1),
    NEW.id_usuario);
END;
```

Operación atómica: transferencia entre billeteras

```
def transferir(conexion, id_origen, id_destino, monto, descripcion="Transferencia"): # [AGREGADO]
    """Debito y credito en una sola transaccion: o pasan los dos o ninguno."""
    with conexion:
        w_origen = _id_wallet(conexion, id_origen)
        w_destino = _id_wallet(conexion, id_destino)
        _mover_saldo(conexion, w_origen, -monto, "TRANSFERENCIA_ENVIADA", descripcion, w_destino)
        _mover_saldo(conexion, w_destino, monto, "TRANSFERENCIA_RECIBIDA", descripcion, w_origen)
```

ATOMICIDAD DE LAS OPERACIONES

El bloque `with conexion` confirma los cambios solo si todas las sentencias se ejecutan sin error; en caso contrario, los revierte. En una transferencia sin saldo suficiente, la restricción `CHECK (saldo >= 0)` rechaza el débito y la base de datos descarta también el crédito, por lo que el dinero no aparece ni desaparece.

Creación de la base de datos

La ejecución de `python3 configurar_db.py` en la carpeta del proyecto genera el archivo `quantum_wallet.db` e imprime el resultado de cada etapa.

```

=====
QUANTUM CORE - Configuracion de la base de datos quantum_wallet.db
=====
[1] SQLite 3.50.4 - archivo: quantum_wallet.db
    PRAGMA foreign_keys = ON

[2] Estructura creada
    Tablas (6):
    - configuracion_sistema
    - empleados
    - transacciones
    - usuarios
    - vinculaciones
    - wallets
    Indices (2):
    - idx_transacciones_wallet_fecha
    - idx_vinculaciones_empleado
    Triggers (4):
    - trg_empleado_valida_tipo
    - trg_transaccion_valida_limite
    - trg_usuario_crea_wallet
    - trg_vinculacion_valida_empresa
    Vistas (3):
    - vista_movimientos
    - vista_nomina
    - vista_saldos

[3] Datos de prueba
    Saldos por usuario (vista_saldos):
    1 Ana Torres PERSONA W-00001 220,000.50 COP
    2 Bancolombia EMPRESA W-00002 300,000.00 COP
    3 Quantum SAS EMPRESA W-00003 2,300,000.00 COP
    4 Laura Gomez EMPLEADO W-00004 4,349,999.50 COP
    5 Pedro Rios EMPLEADO W-00005 3,200,000.00 COP
    Nomina (vista_nomina):
    Quantum SAS -> Laura Gomez Desarrolladora 4,500,000.00
    Quantum SAS -> Pedro Rios Analista 3,200,000.00
    Transacciones registradas: 11
    Conciliacion saldo vs. historial: correcta en todas las wallets
  
```

Figura 3. Ejecución de `configurar_db.py`: versión de SQLite, objetos creados, saldos por usuario, nómina y conciliación de saldos.

```

[4] Pruebas de integridad (todas deben ser RECHAZADAS)
    RECHAZADA Wallet con un propietario que no existe (FK)
    -> IntegrityError: FOREIGN KEY constraint failed
    RECHAZADA Segunda wallet para el mismo usuario (relacion 1 a 1)
    -> IntegrityError: UNIQUE constraint failed: wallets.id_propietario
    RECHAZADA Correo electronico repetido (UNIQUE)
    -> IntegrityError: UNIQUE constraint failed: usuarios.email
    RECHAZADA Empresa registrada sin NIT (CHECK)
    -> IntegrityError: CHECK constraint failed: ck_usuario_nit_empresa
    RECHAZADA Pago mayor al saldo disponible (CHECK saldo >= 0)
    -> IntegrityError: CHECK constraint failed: ck_wallet_saldo_no_negativo
    RECHAZADA Recarga que supera el limite del Banco Central (trigger)
    -> IntegrityError: el monto supera el limite por transaccion
    RECHAZADA Segunda fila de configuracion (Singleton)
    -> IntegrityError: CHECK constraint failed: ck_config_fila_unica
    RECHAZADA Ficha de empleado para un usuario PERSONA (trigger)
    -> IntegrityError: el usuario no es de tipo EMPLEADO
    Resultado: 8 de 8 operaciones invalidas rechazadas

[5] AUTOINCREMENT y borrado en cascada
    Se creo y se borro el usuario 6; wallets huérfanas: 0
    El siguiente usuario recibio el id 7 (el 6 no se reutiliza)
  
```

Figura 4. Continuación de la ejecución: pruebas de integridad y comprobación de AUTOINCREMENT y del borrado en cascada.

INTERPRETACIÓN

El script creó las seis tablas, los dos índices, los cuatro triggers y las tres vistas. Los cinco usuarios recibieron su billetera de forma automática y los saldos coinciden con el historial de once transacciones. La billetera de Ana Torres registra 220.000,50 y la de Laura Gomez 4.349.999,50, lo que confirma que el tipo `REAL` conserva los centavos de la transferencia de 150.000,50. Las ocho operaciones inválidas fueron rechazadas.

Inspección con DB Browser for SQLite

La base de datos generada se abrió con DB Browser for SQLite para verificar su estructura y su contenido de forma independiente del script.

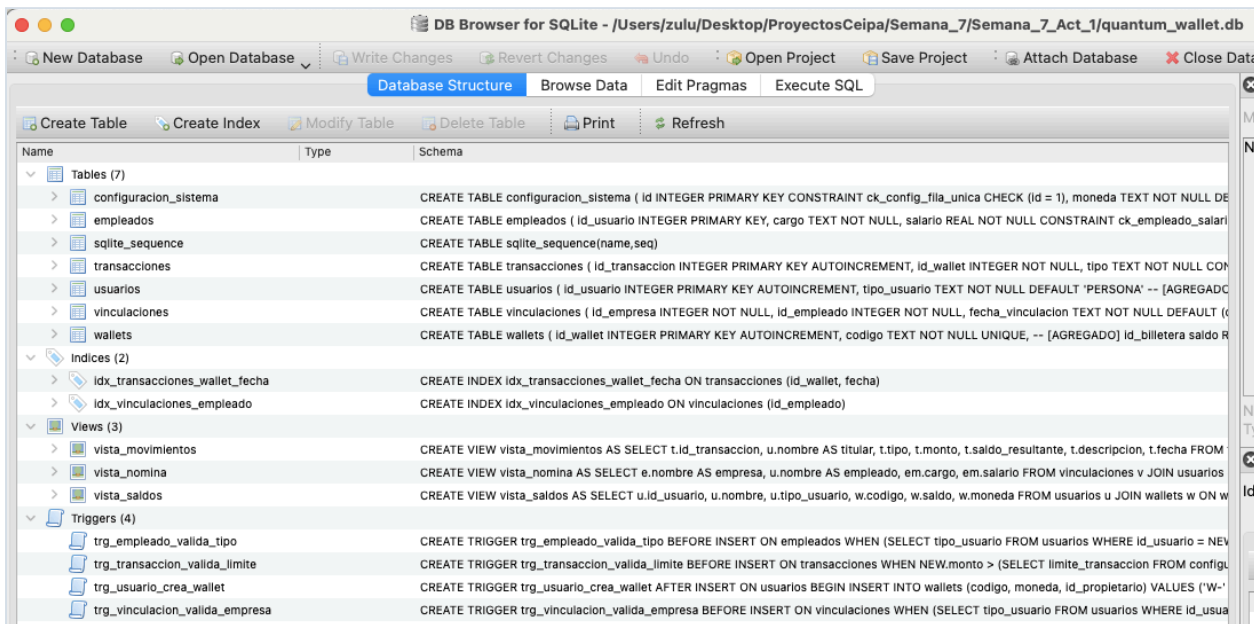


Figura 5. Pestaña Database Structure: tablas, índices, vistas y triggers de quantum_wallet.db, con la ruta del archivo en la barra de título.

La herramienta lista siete tablas porque incluye `sqlite_sequence`, una tabla interna que SQLite crea al usar `AUTOINCREMENT` para registrar el último identificador asignado a cada tabla.

Database Structure Browse Data Edit Pragmas E							
Table: usuarios							
	id_usuario	tipo_usuario	nombre	email	cedula	nit	fecha_registro
	Filter	Filter	Filter	Filter	Filter	Filter	Filter
1	1	PERSONA	Ana Torres	ana@quantum.com	NULL	NULL	2026-10-02 20:51:55
2	2	EMPRESA	Bancolombia	contacto@bancolombia.com	NULL	900123456-7	2026-10-02 20:51:55
3	3	EMPRESA	Quantum SAS	pagos@quantum.com	NULL	901555222-1	2026-10-02 20:51:55
4	4	EMPLEADO	Laura Gomez	laura.gomez@quantum.com	1036123456	NULL	2026-10-02 20:51:55
5	5	EMPLEADO	Pedro Rios	NULL	1017987654	NULL	2026-10-02 20:51:55

Figura 6. Tabla usuarios: tres tipos de usuario, NIT solo en las empresas y cédula en los empleados.

Database Structure Browse Data						
Table: wallets						
<u>id_wallet</u>	codigo	saldo	moneda	fecha_creacion	<i>id_propietario</i>	
Filter	Filter	Filter	Filter	Filter	Filter	
1	1	W-00001	220000.5	COP	2026-10-02 20:51:55	1
2	2	W-00002	300000.0	COP	2026-10-02 20:51:55	2
3	3	W-00003	2300000.0	COP	2026-10-02 20:51:55	3
4	4	W-00004	4349999.5	COP	2026-10-02 20:51:55	4
5	5	W-00005	3200000.0	COP	2026-10-02 20:51:55	5

Figura 7. Tabla wallets: una billetera por usuario, vinculada por la llave foránea *id_propietario* (en cursiva).

Database Structure Browse Data Edit Pragmas Execute SQL								
Table: transacciones								
<u>id_transaccion</u>	<i>id_wallet</i>	tipo	monto	saldo_resultante	<i>id_wallet_contraparte</i>	descripcion	fecha	
Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter	
1	1	1	RECARGA	100000.0	100000.0	<i>NULL</i>	Recarga	2026-10-02 20:51:55
2	2	1	PAGO	30000.0	70000.0	<i>NULL</i>	Pago en comercio	2026-10-02 20:51:55
3	3	2	RECARGA	500000.0	500000.0	<i>NULL</i>	Recarga	2026-10-02 20:51:55
4	4	2	PAGO	200000.0	300000.0	<i>NULL</i>	Pago a proveedor	2026-10-02 20:51:55
5	5	3	RECARGA	10000000.0	10000000.0	<i>NULL</i>	Capital de trabajo	2026-10-02 20:51:55
6	6	3	TRANSFERENCIA_ENVIADA	4500000.0	5500000.0	4	Pago de nomina	2026-10-02 20:51:55
7	7	4	TRANSFERENCIA_RECIBIDA	4500000.0	4500000.0	3	Pago de nomina	2026-10-02 20:51:55
8	8	3	TRANSFERENCIA_ENVIADA	3200000.0	2300000.0	5	Pago de nomina	2026-10-02 20:51:55
9	9	5	TRANSFERENCIA_RECIBIDA	3200000.0	3200000.0	3	Pago de nomina	2026-10-02 20:51:55
10	10	4	TRANSFERENCIA_ENVIADA	150000.5	4349999.5	1	Devolucion de prestamo	2026-10-02 20:51:55
11	11	1	TRANSFERENCIA_RECIBIDA	150000.5	220000.5	4	Devolucion de prestamo	2026-10-02 20:51:55

Figura 8. Tabla transacciones: recargas, pagos, nómina y transferencia con centavos; cada movimiento registra el saldo resultante y la billetera contraparte.

RESULTADO DE LA VERIFICACIÓN

DB Browser for SQLite identifica la llave primaria de cada tabla (subrayada) y las llaves foráneas (en cursiva). Los datos coinciden con la salida del script, y los valores nulos aparecen únicamente donde el modelo los admite: el NIT de las personas, la cédula de las empresas y la contraparte de las recargas y los pagos.

Operaciones inválidas rechazadas

Para comprobar que el esquema protege la información, el script intenta ocho operaciones que violan las reglas del modelo. En todos los casos la base de datos las rechazó con un error de integridad y revirtió los cambios.

OPERACIÓN INTENTADA	MECANISMO	MENSAJE DE SQLITE
Billetera con un propietario inexistente	Llave foránea	FOREIGN KEY constraint failed
Segunda billetera para el mismo usuario	UNIQUE en id_propietario	UNIQUE constraint failed: wallets.id_propietario
Correo electrónico repetido	UNIQUE en email	UNIQUE constraint failed: usuarios.email
Empresa registrada sin NIT	CHECK	CHECK constraint failed: ck_usuario_nit_empresa
Pago mayor al saldo disponible	CHECK	CHECK constraint failed: ck_wallet_saldo_no_negativo
Recarga superior al límite del Banco Central	Trigger	el monto supera el limite por transaccion
Segunda fila de configuración	CHECK (id = 1)	CHECK constraint failed: ck_config_fila_unica
Ficha de empleado para un usuario PERSONA	Trigger	el usuario no es de tipo EMPLEADO

AUTOINCREMENT y borrado en cascada

El script registró un usuario temporal con identificador 6, le hizo una recarga y luego lo eliminó. La consulta posterior no encontró billeteras huérfanas, lo que confirma que `ON DELETE CASCADE` eliminó la billetera y su historial. El siguiente usuario recibió el identificador 7: `AUTOINCREMENT` no reutiliza los identificadores eliminados, lo que evita que un registro nuevo herede referencias de uno anterior. Ambos usuarios temporales se retiraron al finalizar, de modo que los datos de prueba quedan limpios.

SQLite frente a PostgreSQL

SQLite es adecuado para la etapa actual de Quantum Core, pero una Fintech en operación tiene exigencias que conviene anticipar. La comparación siguiente resume los criterios para decidir una migración.

CRITERIO	SQLITE	POSTGRESQL
Arquitectura	Biblioteca embebida; la base es un archivo local.	Servidor independiente con arquitectura cliente-servidor.
Concurrencia	Múltiples lectores, pero un solo escritor a la vez.	Múltiples escritores simultáneos mediante control de concurrencia multiversión (MVCC).
Seguridad	Depende de los permisos del archivo; no maneja usuarios.	Roles, privilegios por tabla y conexiones cifradas.
Valores monetarios	<code>REAL</code> de coma flotante; requiere redondeo en cada operación.	<code>NUMERIC(14, 2)</code> con precisión decimal exacta.
Crecimiento	Adecuado para aplicaciones locales, prototipos y volúmenes moderados.	Replicación, respaldos en caliente y alta disponibilidad.
Administración	Sin instalación ni configuración.	Requiere instalación, configuración y administración del servidor.

CRITERIO DE MIGRACIÓN

SQLite cubre el desarrollo y las pruebas de Quantum Core. Cuando el sistema atienda a varios usuarios que registren transacciones al mismo tiempo, o cuando maneje dinero real que exija precisión decimal exacta y control de acceso, corresponde migrar a PostgreSQL. El esquema diseñado se traslada casi sin cambios: `AUTOINCREMENT` pasa a columnas `GENERATED ALWAYS AS IDENTITY`, `REAL` a `NUMERIC(14, 2)` y los triggers a funciones en PL/pgSQL.

Conclusiones y oportunidades de mejora

La persistencia resuelve la volatilidad de Quantum Core. Los usuarios, las billeteras y los movimientos se conservan en `quantum_wallet.db`, aunque la aplicación se cierre, y el script reconstruye la base de forma idéntica en cada ejecución.

El modelo de clases se tradujo sin pérdida de significado. Cada relación del diagrama UML tiene su equivalente en el esquema: la composición se expresa con `ON DELETE CASCADE`, la agregación con una tabla intermedia que conserva al empleado y la herencia con una llave primaria compartida.

La integridad depende de la base de datos y no solo del código. Las llaves, las restricciones `CHECK` y los triggers rechazaron las ocho operaciones inválidas, incluidas las que se intentaron directamente con SQL, sin pasar por las funciones del script.

La elección de tipos tiene efecto directo sobre el negocio. El tipo `REAL` conservó los centavos de las transferencias; con `INTEGER`, los saldos habrían perdido la parte decimal.

Oportunidades de mejora

MEJORA	DESCRIPCIÓN
Montos en centavos	Almacenar los valores monetarios como enteros en centavos, o como <code>NUMERIC</code> en PostgreSQL, para eliminar el redondeo de coma flotante.
Capa de acceso a datos	Integrar las funciones del script en las clases del modelo, de modo que <code>Usuario</code> y <code>wallet</code> lean y escriban directamente en la base.

REPOSITORIO

El código fuente, la base de datos, el script de consultas, el modelo entidad-relación (fuente `.drawio` y exportaciones PNG y SVG), el README y este informe se encuentran en github.com/DavisZulu/Semana_7_Act_1.

Referencias

- Castro, E. y de Pablo, C. (2013). *Desarrollo de bases de datos: casos prácticos*. RA-MA.
- DB Browser for SQLite. (2026). *DB Browser for SQLite* [Software]. <https://sqlitebrowser.org>
- Hueso, L. (2012). *Gestión de bases de datos* (2.ª ed.). RA-MA.
- Peláez Loaiza, S. (2026). *FUNDAMENTOS-DE-SOFTWARE* [Repositorio de código]. GitHub. <https://github.com/SimonP8/FUNDAMENTOS-DE-SOFTWARE>
- PostgreSQL Global Development Group. (2026). *PostgreSQL documentation*. <https://www.postgresql.org/docs/>
- Python Software Foundation. (2026). *sqlite3: DB-API 2.0 interface for SQLite databases*. The Python Standard Library. <https://docs.python.org/3/library/sqlite3.html>
- SQLite. (2026a). *Datatypes in SQLite*. <https://www.sqlite.org/datatype3.html>
- SQLite. (2026b). *SQLite foreign key support*. <https://www.sqlite.org/foreignkeys.html>
- SQLite. (2026c). *SQLite autoincrement*. <https://www.sqlite.org/autoinc.html>
- W3Schools. (2026). *SQL CREATE TABLE statement*. https://www.w3schools.com/sql/sql_create_table.asp